

1 LOGIC GATES

1.1 Prerequisite theoretical knowledge

Truth tables of AND, OR, NOT, NAND, NOR, EX-OR, EX-NOR logic gates, their logic equations and symbols. Propagation delay. Maximum and minimum allowable input-output voltage and current limits, fan-out capability.

1.2 Work execution

1.2.1 AND gate

Using 1/4 of the 7408 integrated circuit (figure 1), connect logic levels 0 and 1 to inputs A and B, a LED indicator to the output (figure 2) and complete the truth table for the two-input AND gate (table 1).

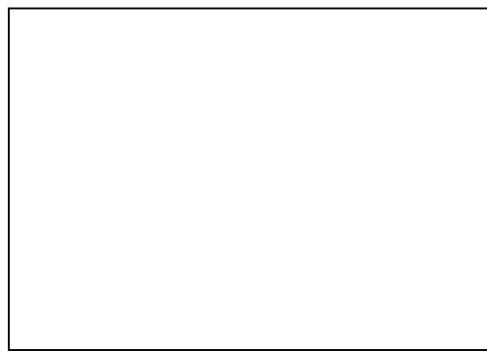
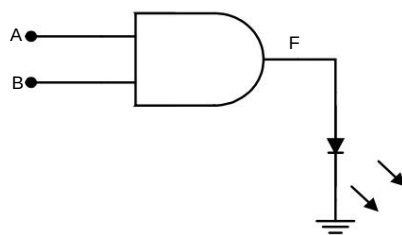


Fig. 1 Terminals of the 7408 integrated circuit



A	B	F	
0	0		
0	1		
1	0		
1	1		

Fig. 2. Measurements on an AND gate Table 1 Gate truth table
Two-input AND

From the truth table, extract the logic equation for the gate
AND

$$F =$$

Note: The 74 & 54 series TTL gates are positive-bias gates logic and operate with TTL level voltages as listed. Specifically: $V_H = 5V$ <<Logical 1>> and $V_L = 0V$ <<Logical 0>>. The supply $V_{cc} = 5V$ and $Gnd = 0V$.

1.2.2 OR, NAND, NOR, EX-OR gates

Repeat paragraph 1.2.1 for the OR EX-OR gates , NAND , NOR and using the 7432, 7400, 7402 and 7486 integrated circuits. respectively. Complete the truth table for each and extract the corresponding logic equations. Figure 3 shows the terminals of the corresponding integrated circuits 7432, 7400, 7402 and 7486.

Fig. 3 Integrated circuit terminals
of gates 7432, 7400, 7402 and 7486

1.2.3 The open collector EX-NOR gate (74266).

Recall that in a gate with an open collector output the output It "comes into operation" when connected to a resistor in the power supply. This resistor is called a pull-up. The 74266 is a quad EX-NOR two-input gate with open collector output (figure 4). Repeat the paragraph 1.2.1 and for the EX-NOR gate by adding a 10k pull-up resistor from the output to the 5V supply (figure 5).

Fig. 4 Terminals of the 74266 integrated circuit

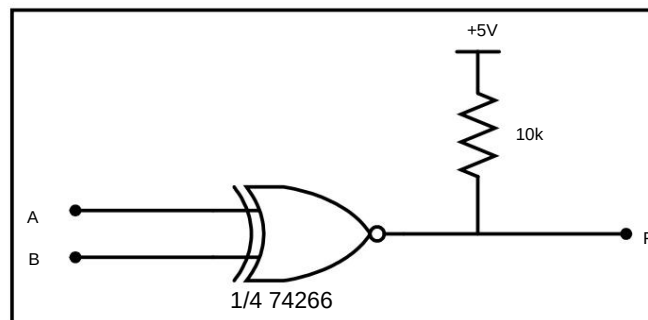
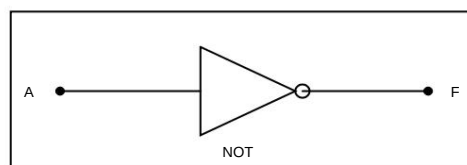


Fig. 5 Pull-up resistor connection to an open collector EX-NOR gate

1.2.4 Inverter - NOT Gate

Use 1/6 of the 7404 integrated circuit (figures 6 and 7)
Complete the truth table of the NOT gate (table 2).

Fig. 6 Terminals of the 7404 integrated circuit



AF	
0	
1	

Fig. 7 Inverter

Table 2 NOT gate truth table

1.2.5 Control of digital signals by gateways.

a) If in an AND, OR gate, NAND, Consider a two-input NOR input A as a control signal that takes values "0" and "1" and input B as a digital signal controlled by the gate then in general the output F of the gate sometimes has a value equal to B or B or "1" or "0" (figure 8).

Below are the values of the output F of each gate when A=0 and A=1

AND gate:

$A = 0 \Rightarrow F = 0$

$A = 1 \Rightarrow F = B$

OR gate:

$A = 0 \Rightarrow F = B$

$A = 1 \Rightarrow F = 1$

NAND gate:

$A = 0 \Rightarrow F = \overline{B}$

$A = 1 \Rightarrow F = \overline{1} = 0$

NOR gate:

$A = 0 \Rightarrow F = \overline{B}$

$A = 1 \Rightarrow F = \overline{1} = 0$

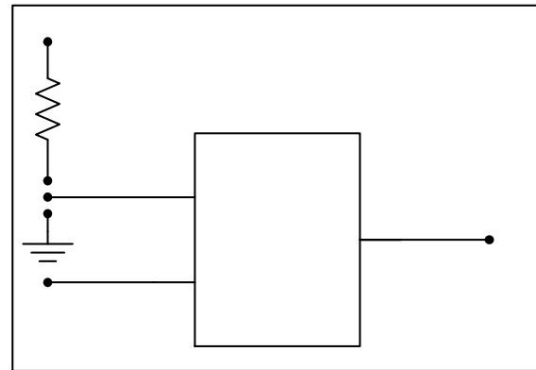


Fig. 8 Digital signal control
from gates

b) If in an EX-OR or EX-NOR gate we consider input A as control signal then the output takes values equal to B or the gates

EX-OR or EX-NOR operate as inverters for a certain value of control signal or leave input B as it is at the output.

EX-OR gate:

$A = 0 \Rightarrow F = B$

$A = 1 \Rightarrow F = \overline{B}$

EX-NOR Gate:

$A = 0 \Rightarrow F = \overline{B}$

$A = 1 \Rightarrow F = B$

c) The EX-NOR gate is also referred to as a comparator because its output is "1" when both inputs are "0" or both are "1".

So if $A \neq B \Rightarrow F = 0$

and if $A = B \Rightarrow F = 1$

Verify the above from the truth tables of the gates and from their logic equations.

1.2.6 Propagation Delay.

Propagation delay "propagation delay" is the average time required for the signal at the output of a gate to change state after the signal at the input changes. The propagation delay determines the maximum operating speed of the gate and digital circuits in general. Two delay times are defined: The delay time when the output goes from low to

high level t_{PLH} and the delay time when the output goes from high to low level t_{PHL}

The average delay time t_{PAV} is obtained from the average of the t_{PLH}

and t_{PHL} . These times are measured between the points where the signal is located at the midpoint between the low and high levels (figure 9).

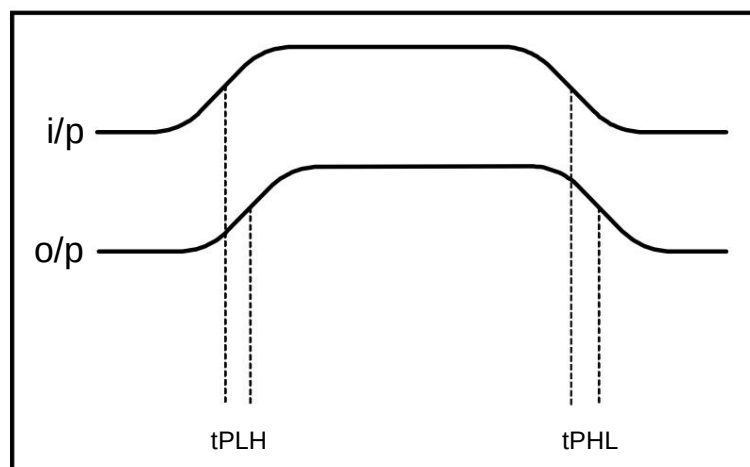


Fig. 9 Delay time measurements.

Delay times are measured in ns and are given in data books of the manufacturers.

Make the circuit of figure 10 by connecting 4 gates AND in series. At the input of the first gate, apply a square pulse of frequency 100 kHz. Observe the output on an oscilloscope, in time correspondence with the input with as short times as possible in the horizontal scan of the oscilloscope for better analysis. Measure and note the times t_{PLH} and t_{PHL} that concern the 4 AND gates in series, so for each gate we have:

$$t_{PLH} \quad \frac{t_{PLH}}{4} \quad t_{PHL} \quad \frac{t_{PHL}}{4}$$

and the average delay time is:

$$t_{PAV} = \frac{t_{PHL} + t_{PLH}}{2}$$

With the measurements you took, complete table 3.

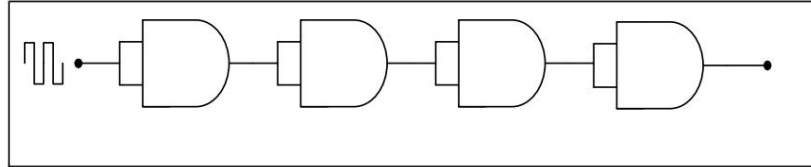


Fig. 10 Propagation delay in an AND gate

tPLH	n.s.
tPHL	n.s.
t PLH	n.s.
t PHL	n.s.
t PAV	n.s.

Table 3 Propagation delay measurements

1.2.7 Maximum and minimum output voltage values and maximum Output currents. Driving ability (fan-out).

The "H" index in the output currents and voltages implies that the value refers to a "High" output state, while the "L" indicator implies that this value is reported for output state "Low". Referring to positive logic in the following the following values given are important from manufacturers:

The maximum value of the output current in High state IOH (max) determines the maximum output current that can flow from the output for the which the output voltage is still recognized as a logical "1" The value that of the output voltage is defined as VOH (min) and is the minimum allowed output voltage for "High" output - logical "1". The IOH has conventionally negative sign because it points outwards from the gate.

The maximum value of the output current when it is in Low IOL state determines the maximum current that can flow to the output for which the output voltage is still recognized as a logical "0". The This voltage value is defined as VOL (max) and is the maximum allowed output voltage for "Low" output - logical "0". IOL has a positive sign conventionally, because it is current flowing towards the gate.

Fan-out is defined as the maximum number of inputs of similar gates that can be connected to the output of a gate without affecting the normal operation. Obviously for normal operation the voltages and currents gate exit must remain within the limits specified from the maximum and minimum values as appropriate.

The standard TTL load for TTL circuits specifies the amount of current required for a gate input to be driven so that the driving capacity can be given e.g. as 10 TTL standard loads.

1.2.8 Gateway overload

Connect the following circuit of figure 11. Determine by setting the input to 0 and 5 V that after two inverters the output follows the input. This is the normal operation of the circuit.

Then connect the input to logic "0" and a 33 Ω resistor from point A to earth. What do you observe at the output of the circuit and at the voltage at point A ($V_A = ?$).

Disconnect the 33 Ω resistor from the circuit and connect it input at logic "1" and a 180 Ω resistor from point A to 5V. What observe at the output and the voltage at point A ($V_A = ?$).

The voltage at point A remains at normal values with the addition of the resistors that overload the gate? Where are the external normal operating values in the circuit output voltage?

We note that the output voltage and current limit values for the 74LS04 inverter are as shown in table 4.

IOH max	-400 mA
IOL max	8mA
VOH min	2.7V
VOL max	0.5V

Table 4 Output voltage and current limits for the 74LS04

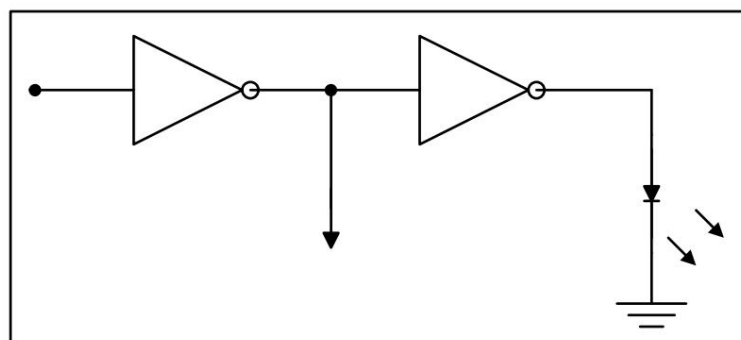


Fig. 11 Gate overload measurements

2 BOOLE ALGEBRA - KARNAUGH MAP - DESIGN ME

NAND AND NOR.

2.1 Theoretical part

2.1.1 Boolean Algebra

Definition

In logic circuits, variables can only take two values. values the logical "1" (true) and the logical "0" (false). Boolean algebra and in fact the two-valued Boolean algebra as presented by Shannon - as switch algebra - is very useful in designing logic circuits. Briefly, Boolean algebra (meaning the two-valued one from now on) is forward) can be defined as follows:

Boolean algebra is defined for a set of logical variables x, y, z , ... which take only two values "0" or "1" and where the following apply between them actions:

1. Logical "OR" ($\vee$) which is denoted by 2. $+$
- Logical "AND" ($\wedge$) which is denoted by 3. $\cdot$
- Logical inversion "NOT" ($\neg$) which is denoted by 4. $-$
- Finally, the relation of equality applies.

Dualism

A very important property of Boolean algebra is duality. (duality principle). Duality means that every algebraic expression that is based on the axioms of Boolean algebra and is also valid in another form, the its binary where the operations (OR) and (AND) will alternate with each other and the "0" and "1" will also alternate with each other. So almost all the axioms and theorems of Boolean algebra have two forms which are between dual.

Boolean algebra axioms

Like any algebra, Boolean algebra is based on certain axioms and then theorems based on the axioms.

1. The result of AND, OR and NOT operations between variables also have a value of "0" or "1".
2. $x + 0 = x$ and its dual $x \cdot 1 = x$
3. $x + y = y + x$ and its dual $x \cdot y = y \cdot x$
4. $x \cdot (y + z) = x \cdot y + x \cdot z$ and its dual $x + y \cdot z = (x + y) \cdot (x + z)$ —
5. $x + x = 1$ and its dual $x \cdot x = 0$

Theorems of BOOLE algebra

The theorems of BOOLE algebra are the following:

1. $x + x = x$ and its dual $x \cdot x = x$ 2. x

$+1 = 1$ and its dual $x \cdot 0 = 0$

3. $\overline{\overline{x}} = x$

4. $x + (y \cdot z) = (x + y) \cdot (x + z)$ and its dual

$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$

5. De Morgan's Theorems: $\overline{xy} = \overline{x} + \overline{y}$ and its dual $\overline{\overline{xy}} = xy$

$\overline{\overline{xy}} = xy$

6. Absorption theorem: $x + x \cdot y = x$ and its dual $x \cdot (x + y) = x$

2.1.2 Karnaugh Map – Simplification

The simplification of the logic equation resulting from the truth table describing a logic problem is a very important step in the design of the corresponding logic circuit because it reduces to a minimum its complexity both in terms of the number of components and in terms of the connections between them. Although it is possible to simplify logic equations algebraically, for logic equations of up to six variables the most suitable way is the graphical simplification method with the Karnaugh map. Karnaugh maps with fewer variables are simpler while for more they are more complex and in the case of more than six variables practically inapplicable, so the simplification of such logic functions is done with the help of special programs on computers. The Karnaugh map of four variables is depicted in Figure 1.

CD \ AB	00	01	11	10	
00				1	
01			1	1	
11			1	1	
10					

Fig. 1 4-variable Karnaugh map

Note in figure 1 the order of the combinations of AB, CD as indicated on the vertical and horizontal sides. The order of the combinations follows the Gray code! Each of the 16 squares of the map corresponds to a single combination out of the 16 variables A, B, C, D

while in each adjacent square only one variable is changed horizontally or vertically. The representation of a logical function in the Karnaugh map will be done either by the truth table or by the logical equation and each term of a logical function that is in the form of a sum of minimizers is depicted with "1" in the corresponding square. The simplification in the Karnaugh map is based

on the creation of groups (sub-papers) of adjacent aces, whose number can be two, four or even eight for the four variables. Adjacent aces are also considered to be those located in extreme columns or in extreme rows, because the Karnaugh can be considered as an expansion of the "sphere" that would be created by the contact of the extreme columns between them and the extreme rows between them.

Our goal is to create large groups and include all aces once in a group if possible, while there is a case where some aces are included in more than one group. When simplifying the logical equation, variables are eliminated depending on the size of each group of aces formed. In the group of two aces, one variable is eliminated, in the group of four aces, two variables, and in the group of eight aces, three variables.

Thus in figure 1 the two aces will eliminate the variable B giving the term: ACD . Indeed the two aces had resulted from the terms $ABCD$ and $\overline{A}BCD$. But $ABCD + \overline{A}BCD = \overline{A}CDB = ACD$. By extension prove that the group of four aces will eliminate the variables A and C and will give the term BD . Therefore the logical function: $F = ABCD + \overline{A}BCD + \overline{A}B\overline{C}D + \overline{A}B\overline{C}\overline{D}$ represented by the five aces in the Karnaugh map of figure 1 will be simplified to:

$$F = ACD + BD$$

Example 1

The following truth table is given and the corresponding function is asked to be simplified with a Karnaugh map.

A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	1	0	0
0	1	0	1	0
0	1	1	1	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

From the truth table we have the logical equation:

$$F = ABCD + \overline{A}BCD + ABCD + \overline{A}B\overline{C}D + \overline{A}BC\overline{D} + \overline{A}BCD + \overline{A}BCD$$

Then we create the Karnaugh map (figure 2), for the above logical equation:

Fig. 2 Karnaugh map of example 1

From the Karnaugh map after creating the three groups of ones we have the following terms for the simplified logical equation:

Of the four ones in the corners the term $\overline{B}\overline{D}$.

four ones formed by the two ones on top and both bottom left the term $\overline{B}C$ and finally

From the pair of ones the term $\overline{A}CD$. So

finally the simplified function that results is:

$$F = \overline{B}\overline{D} + \overline{B}C + \overline{A}CD$$

2.1.3 Simplification with zeros

In the squares of the Karnaugh table where we do not place ones, logical function has a value of zero. So if we were to place "0" in all the squares where one function F is not true the Karnaugh map with the zeros will represent the complementary function $\bar{F}$.

In the case where the zeros are fewer than the ones it is often useful to find the simplified function for $\bar{F}$. This will be done by forming groups (submaps) for neighboring zeros of which - in a similar way to what we did for the ones - give us simplified expression for $\bar{F}$. Then applying the theorem De Morgan we take F in the form of a product of maxima.

Example 2

$F = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}BC\bar{D} + \bar{A}B\bar{C}D + \bar{A}BCD + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D} + A\bar{B}CD + A\bar{B}\bar{C}D + AB\bar{C}\bar{D} + ABC\bar{D} + AB\bar{C}D + ABCD$ and is asked to simplify.

We create the Karnaugh map and represent F with the ones that correspond to its terms and then we place zeros in the remaining empty squares (figure 3).

CD \ AB	00	01	11	10
00	1	0	0	0
01	1	0	1	1
11	1	0	1	1
10	1	0	1	1

Fig. 3 Karnaugh map of example 2

Then we create two groups of zeros (which is very less than the ones) and we get a simplified expression for the complementary function $\bar{F}$. Thus we have $\bar{F} = \bar{C}\bar{D} + \bar{A}BC$ and applying De Morgan after inversion we have:

$$\begin{aligned}\bar{\bar{F}} &= \overline{\bar{C}\bar{D} + \bar{A}BC} \\ \bar{F} &= \overline{\bar{C}\bar{D}} \cdot \overline{\bar{A}BC} \\ F &= (CD)(ABC)\end{aligned}$$

This form of F is called the product of maxima, while the form that resulting from simplification with ones is called the sum of minimums.

2.1.4 Indifferent terms

When in a logical problem in some of the combinations of inputs we don't care about the output value (whether it will be "0" or "1") then these terms are called unimportant terms and in the Karnaugh map as well as in the truth table they are symbolized by "X".

Since the output value for the indifferent terms can be considered either "0" or "1" they can be combined with adjacent "1"s if simplification is done with "1" or with adjacent "0" if simplification is done with "0" and in this way we have the possibility to create more and larger groups resulting in better simplification.

Example 3

Suppose a BCD counter counts decimals. numbers from 0 to 9 and we want to have an output "1" only for decimal "9". The simplified logical function is requested.

The truth table for the above problem is given below. We note that the input combinations for after the decimal "9" will not never appear because the BCD counter after 9 will take the value 0 starting the measurement from the beginning.

So the six combinations that do not occur can be considered as indifferent terms, to be symbolized by "X" which, as we said, indicates that they can have a value of "1" or "0".

	DC	B	A	F		
	00	00	00			
	00	01	00			
	00	10	00			
	00	11	00			
	01	00	00			
	01	01	00			
	01	10	00			
	01	11	00			
	10	00	00			
	10	01	10	10	X	1
	10	11	1	X		
	1		100	X		
1			101	X		
1			110	X		
1			111	X		

From the truth table we draw the Karnaugh table for problem by placing the ace and the indifferent terms (figure 4).

BA \ DC	00	01	11	10
00				
01				
11	X	XXX		
10		1	XX	

Fig. 4 Karnaugh map of example 3

The group that is formed gives us $F = D \cdot A$ while without them indifferent terms, no simplification was possible.

2.1.5 NAND and NOR gates in logic circuit design.

NAND and NOR gates are very easily implemented in integrated circuits and are thus the basic gates used in the design of logic circuits. As we will see below, any logic equation can be

transformed into a form that is implemented only with NAND gates or only with NOR gates. This transformation is easily carried out algebraically with De Morgan's theorems. Let us consider a function in the form of a sum of minima:

$$F = AB\overline{C}\overline{D}$$

By inverting F twice and applying the De theorem Morgan we have:

$$\overline{\overline{F}} = \overline{\overline{AB\overline{C}\overline{D}}}$$

$$F = (\overline{AB})(\overline{\overline{C}\overline{D}})$$

This form of F is implemented with NAND gates only, as shown in figure 5 where we observe that instead of inverters for the B terms, $\overline{NAND}$, $\overline{C}$ gates with their inputs short-circuited are used.

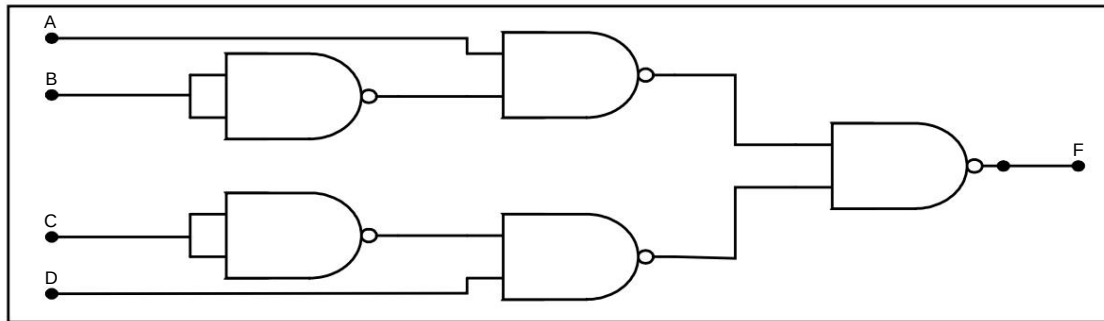


Fig. 5 Implementation of $F = \overline{A} \overline{B} \overline{C} \overline{D}$ with NAND gates

If in the function F we place a double negation on each minimum and applying De Morgan's theorem we will have:

$$\begin{aligned}
 F &= \overline{A} \overline{B} \overline{C} \overline{D} \\
 F &= \overline{\overline{A} \overline{B} \overline{C} \overline{D}} \\
 F &= \overline{\overline{A} \overline{B} \overline{C} \overline{D}} \quad \text{and with double negation } F = \overline{\overline{\overline{A} \overline{B} \overline{C} \overline{D}}} = \overline{\overline{A} \overline{B} \overline{C} \overline{D}}
 \end{aligned}$$

This form of F is implemented with NOR gates only (figure 6), where we notice that here instead of inverters, NOR gates with short-circuited inputs are used.

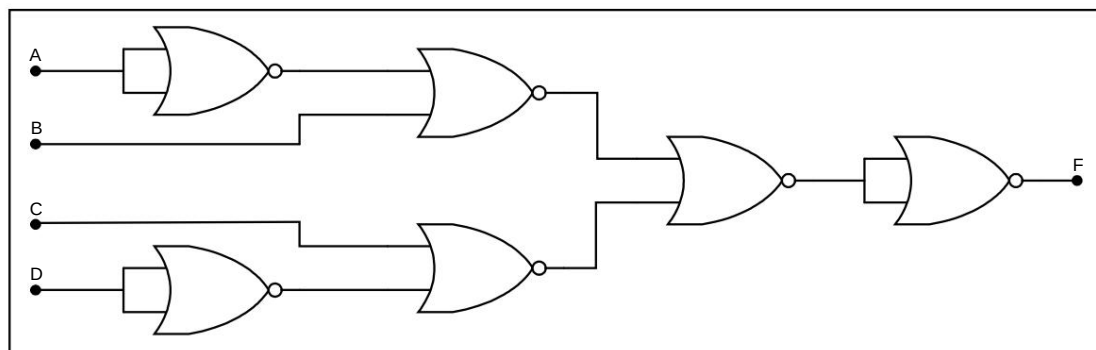


Fig. 6 Implementation of $F = \overline{A} \overline{B} \overline{C} \overline{D}$ with NOR gates

The function is now given

$$F = (AB)(\overline{C} \overline{D}) \text{ which is expressed in terms of a product of maxima.}$$

Inverting F twice we have

$$F = \overline{\overline{(AB)(\overline{C} \overline{D})}} \quad \text{and} \quad F = \overline{\overline{(AB)(\overline{C} \overline{D})}}$$

This form is implemented with NOR gates only (figure 7).

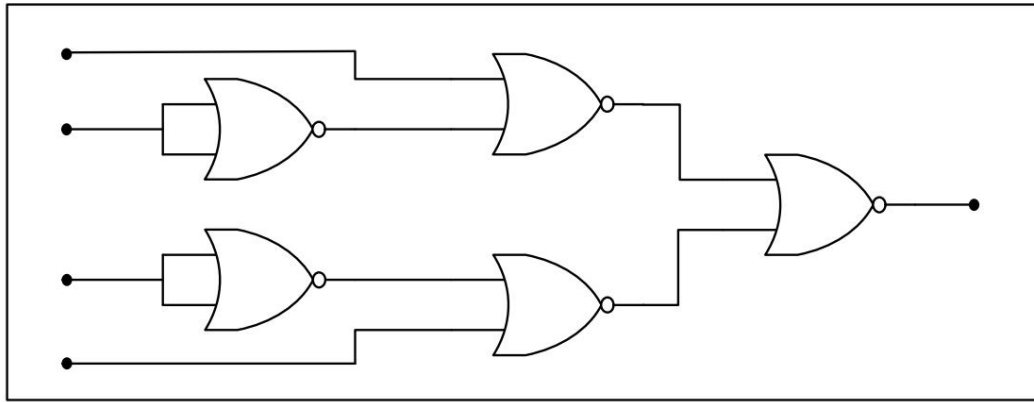


Fig. 7 Implementation of $F = (AB)(CD)$ with NOR gates

However, if we placed a double inversion in each maximin in the same function, we would have:

$$\begin{aligned}
 F &= (AB)(\overline{CD}) \\
 F &= \overline{\overline{(AB)(\overline{CD})}} \\
 F &= \overline{(AB)(\overline{CD})} \text{ and with double negation in } F \\
 F &= \overline{\overline{(AB)(\overline{CD})}}
 \end{aligned}$$

This format is implemented with NAND gates (figure 8)

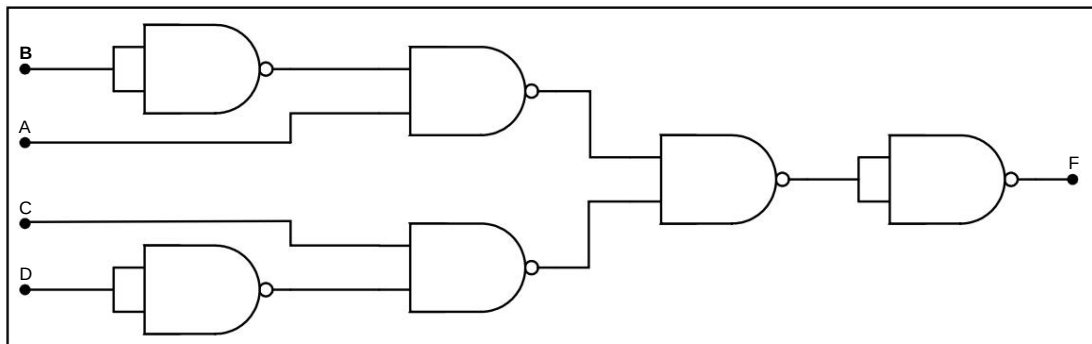


Fig. 8 Implementation of $F = (AB)(CD)$ with NAND gates

We observe that for a function in the form of a sum of minimizers, the implementation with NAND gates is easier and with one less logic level than in the implementation with NOR gates. On the contrary, for a function in the form of a product of maximizers, the implementation with NOR gates is easier and with one less logic level than in the implementation with NAND gates. Thus, usually after simplification with a Karnaugh map based on ones, the simplified function in the form of a sum of minimizers is transformed so that it can be implemented with NAND gates, while if the simplification was based on zeros, the simplified function in the form

product of magisters is transformed so that it can be implemented with gates NOR. In fact, a comparison is made of the two alternative solutions and the one that is simpler and of course more economical of the two is chosen.

Example 4

Given the following Karnaugh map (figure 9). Investigate which implementation is more economical, i.e. with NAND or NOR gates, and design the logic circuit of the proposed implementation. Initially, we create the groups of "1" which give us the logical function:

$$F = \overline{D}C AB \text{ and for NAND gates}$$

$$F = \overline{\overline{F}} = \overline{\overline{D}C AB}$$

$$F = \overline{DC AB}$$

CD \ AB	00	01	11	10
00	1	0	1	1
01	1	0	X	1
11	1	0	X	1
10	1	1	1	1

Fig. 9 Karnaugh map of example 4

Then we create the groups with the zeros and we have in order:

$$F = \overline{A} \overline{C} D BD$$

$$F = \overline{A} \overline{C} D BD$$

$$F = \overline{(AC D)(BD)}$$

$$F = (ACD)(BD) \text{ and for NOR gates:}$$

$$F = \overline{\overline{F}} = \overline{\overline{(ACD)(BD)}} \quad F = \overline{(ACD)(BD)}$$

We see that the implementation with NAND gates along with the inversions of the terms C, B will require four NAND gates, one of which will be three-input, while the implementation with NOR gates along with the inversions of D and B will require five NOR gates, one of which will be three-input. We choose the design with NAND gates and the resulting logic circuit is shown in figure 10.

How could we use two-input NAND gates?
throughout the circuit? What would be the disadvantage of such a design?

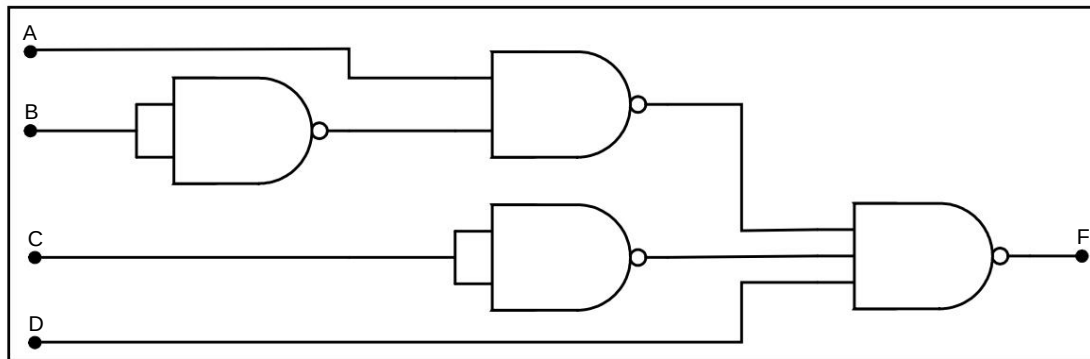


Fig. 10 Implementation of the function of example 4 with NAND gates

2.2 Work execution

1. Using a NOR gate, two NOT gates and an AND gate
Connect two appropriate logic circuits to verify the
De Morgan's theorem $\overline{XY} = \overline{X} \overline{Y}$. Connect the inputs X, Y to both
circuits, on the same switches and fill in from your measurements
the following truth table that will verify the theorem.

Entrances		Exit a Exit b	
X	Y	$\overline{XY}$	$\overline{X} \overline{Y}$
0	0		
0	1		
1	0		
1	1		

2. Using an OR gate and an AND gate
Connect a logic circuit that verifies the power of
absorption theorem. $X.(X + Y) = X$. Complete from the measurements
you the following truth table that will verify the theorem.

Entrances		Exit
XY	$X.(X + Y)$	
0	0	
0	1	
1	0	
1	1	

3. Design the logic circuits you would need for the verification of the two dual forms of these theorems, namely:

_____ — — —
 $XYXY$ (De Morgan) and
 $X+X \cdot Y = X$ (absorption.).

4. The truth table for a combinational logic circuit is given. which will have four inputs A, B, C, D and one output F. Output F is required to be 1 when input A is 1 and at the same time input B is 0 except the case where $A=1$ and $B=C=D=0$. Also the output F should be 1 when input B is 1 and at the same time one of the two inputs C and D and the two is 1.

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	1	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Does the above truth table represent the logic circuit that did we describe?

With a Karnaugh map, simplify first with ones and then with zeros. Do not proceed to the next step unless you arrive at the correct answers which are: $F = BD+BC+AD+AC$ for simplification with ones and $F = (A+B) \cdot (C+D)$ to simplify with zeros.

Transform the simplified relations so that you can draw two equivalent circuits for the desired logic circuit. One with gates NAND from the simplification with ones and one with NOR gates from the simplification with zeros.

What is the most economical circuit in terms of number of gates? but also integrated ones required for its implementation?

Make the circuit that of the two that can realized with integrated two-input NAND or NOR gates only.

Check the circuit operation by verifying the board based on your measurements? Is the circuit you built the requested?

5. The logical equation is given:

$F = \overline{A}\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}\overline{D} + \overline{A}BCD + \overline{A}BC\overline{D} + \overline{A}B\overline{C}D + \overline{A}B\overline{C}\overline{D}$ with five their indifferent terms: $\overline{A}BC\overline{D}$, $\overline{A}B\overline{C}\overline{D}$, $\overline{A}B\overline{C}D$ and $\overline{A}B\overline{C}\overline{D}$.

Simplify with aces and zeros, draw the corresponding circuits with only NAND gates and with only NOR gates. Implement the more economical of the two. Write the truth table for the above logical equation and verify it experimentally by taking measurements at circuit you connected..

